

Semaphores serve as a single primitive for all things related synchronization.

THE CRUX: HOW TO USE SEMAPHORES

How can we use semaphores instead of locks and condition variables? What is the definition of a semaphore? What is a binary semaphore? Is it straightforward to build a semaphore out of locks and condition variables? To build locks and condition variables out of semaphores?

Semaphores: A Definition

A semaphore is an object with an integer value that we can manipulate with two routines;

`sem_wait()` & `sem_post()`

As the initial value of the semaphore determines its behaviour, first we must initialize it to some value as:

```
1  #include <semaphore.h>
2  sem_t s;
3  sem_init(&s, 0, 1);
```

In above code, we declare a semaphore `s` & initialize its value to 1 by passing 1 in as third argument.

The second argument to `sem_init()` set to 0 & indicates that semaphore is shared b/w threads in same process.

Setting it to a different value indicates that the semaphore is shared b/w processes.

The behaviour of `sem_wait()` or `sem_post()`

```
1  int sem_wait(sem_t *s) {
2      decrement the value of semaphore s by one
3      wait if value of semaphore s is negative
4  }
```

```

6  int sem_post(sem_t *s) {
7      increment the value of semaphore s by one
8      if there are one or more threads waiting, wake one
9  }

```

Salient aspects:

- `sem_wait()` will either return right-away, or it will cause the caller to suspend execution waiting for a subsequent post.
- `sem_post()` does not wait for some condition like `sem_wait()`, it simply increments the value of the semaphore & then, if there is a thread to be woken, wakes one of them.
- The value of semaphore, when negative, is equal to number of threads waiting.
- Actions `sem_wait()` & `sem_post()` perform are atomic.

Binary Semaphores (Locks)

```

1  sem_t m;
2  sem_init(&m, 0, 1); // init to X; what should X be?
3
4  sem_wait(&m);
5  // critical section here
6  sem_post(&m);

```

By setting the initial value 1, semaphores act like locks.

Imagine two threads, `thread0` calls `sem_wait()`; it will first decrement the value of semaphore to 0.

Then, it will only wait if value is not greater than or equal to 0.

As the value is 0, `sem_wait()` will return & the calling thread will enter the critical section.

If no other thread tries to acquire the lock while `thread0` is inside the critical section, when it calls

sem_post(), it will increment value to 1.

Value of Semaphore	Thread 0	Thread 1
1		
1	call sem_wait()	
0	sem_wait() returns	
0	(crit sect)	
0	call sem_post()	
1	sem_post() returns	

Figure 31.4: Thread Trace: Single Thread Using A Semaphore

If thread₀ has not yet called sem_post() & another tries to enter the critical section by calling sem_wait(). In this case, thread₁ will decrement semaphore value to -1, and thus wait.

When thread₀ runs again & calls sem_post(), incrementing value back to 0, then wake the waiting thread₁, which will then acquire the lock.

When thread₁ finishes, it will increment semaphore value to 1.

Val	Thread 0	State	Thread 1	State
1		Run		Ready
1	call sem_wait()	Run		Ready
0	sem_wait() returns	Run		Ready
0	(crit sect begin)	Run		Ready
0	Interrupt; Switch→T1	Ready		Run
0		Ready	call sem_wait()	Run
-1		Ready	decr sem	Run
-1		Ready	(sem<0)→sleep	Sleep
-1		Run	Switch→T0	Sleep
-1	(crit sect end)	Run		Sleep
-1	call sem_post()	Run		Sleep
0	incr sem	Run		Sleep
0	wake(T1)	Run		Ready
0	sem_post() returns	Run		Ready
0	Interrupt; Switch→T1	Ready		Run
0		Ready	sem_wait() returns	Run
0		Ready	(crit sect)	Run
0		Ready	call sem_post()	Run
1		Ready	sem_post() returns	Run

Figure 31.5: Thread Trace: Two Threads Using A Semaphore

When semaphore is used as a lock, it is called binary semaphore as locks have two states (held or not).

Semaphores For Ordering

Semaphores are also useful to order events in a concurrent program.

For example, a thread may wish to wait for a list to become non-empty, so it can delete an element from it.

We are thus using semaphore as an ordering primitive like condition variable.

```
1  sem_t s;
2
3  void *child(void *arg) {
4      printf("child\n");
5      sem_post(&s); // signal here: child is done
6      return NULL;
7  }
8
9  int main(int argc, char *argv[]) {
10     sem_init(&s, 0, 0); // what should X be?
11     printf("parent: begin\n");
12     pthread_t c;
13     Pthread_create(&c, NULL, child, NULL);
14     sem_wait(&s); // wait here for child
15     printf("parent: end\n");
16     return 0;
17 }
```

Figure 31.6: A Parent Waiting For Its Child

As such, the initial value is set to 0.

There are two cases to consider:

1. Parent creates child, and continues to run. In this case, it will call `sem_wait()`, decrement the semaphore to -1, then waits.

When child finally runs, it will call `sem_post()`,

increment the semaphore to 0, and wake parent up, which will then return from sem_wait() & finish the program.

Val	Parent	State	Child	State
0	create (Child)	Run	(Child exists, can run)	Ready
0	call sem_wait()	Run		Ready
-1	decr sem	Run		Ready
-1	(sem<0) →sleep	Sleep		Ready
-1	Switch→Child	Sleep	child runs	Run
-1		Sleep	call sem_post()	Run
0		Sleep	inc sem	Run
0		Ready	wake (Parent)	Run
0		Ready	sem_post() returns	Run
0		Ready	Interrupt→Parent	Ready
0	sem_wait() returns	Run		Ready

Figure 31.7: Thread Trace: Parent Waiting For Child (Case 1)

2. When child runs before parent called sem_wait(). In this case, the child will first run & call sem_post(), increment semaphore value to 1.

When parent runs, it will decrement semaphore value to 0 & return from sem_wait().

Val	Parent	State	Child	State
0	create (Child)	Run	(Child exists; can run)	Ready
0	Interrupt→Child	Ready	child runs	Run
0		Ready	call sem_post()	Run
1		Ready	inc sem	Run
1		Ready	wake (nobody)	Run
1		Ready	sem_post() returns	Run
1	parent runs	Run	Interrupt→Parent	Ready
1	call sem_wait()	Run		Ready
0	decrement sem	Run		Ready
0	(sem≥0) →awake	Run		Ready
0	sem_wait() returns	Run		Ready

Figure 31.8: Thread Trace: Parent Waiting For Child (Case 2)

ASIDE: SETTING THE VALUE OF A SEMAPHORE

We've now seen two examples of initializing a semaphore. In the first case, we set the value to 1 to use the semaphore as a lock; in the second, to 0, to use the semaphore for ordering. So what's the general rule for semaphore initialization?

One simple way to think about it, thanks to Perry Kivolowitz, is to consider the number of resources you are willing to give away immediately after initialization. With the lock, it was 1, because you are willing to have the lock locked (given away) immediately after initialization. With the ordering case, it was 0, because there is nothing to give away at the start; only when the child thread is done is the resource created, at which point, the value is incremented to 1. Try this line of thinking on future semaphore problems, and see if it helps.

The Producer/Consumer (Bounded Buffer) Problem

First attempt

We solve the problem by using two semaphores, empty and full, which the threads will use to indicate when a buffer entry has been emptied or full.

```
1  int buffer[MAX];
2  int fill = 0;
3  int use = 0;
4
5  void put(int value) {
6      buffer[fill] = value;    // Line F1
7      fill = (fill + 1) % MAX; // Line F2
8  }
9
10 int get() {
11     int tmp = buffer[use];    // Line G1
12     use = (use + 1) % MAX;    // Line G2
13     return tmp;
14 }
15
16 sem_t empty;
17 sem_t full;
18
19 void *producer(void *arg) {
20     int i;
21     for (i = 0; i < loops; i++) {
22         sem_wait(&empty);    // Line P1
23         put(i);              // Line P2
24         sem_post(&full);     // Line P3
25     }
26 }
27
28 void *consumer(void *arg) {
29     int tmp = 0;
30     while (tmp != -1) {
31         sem_wait(&full);     // Line C1
32         tmp = get();         // Line C2
33         sem_post(&empty);    // Line C3
34         printf("%d\n", tmp);
35     }
36 }
37
38 int main(int argc, char *argv[]) {
39     // ...
40     sem_init(&empty, 0, MAX); // MAX are empty
41     sem_init(&full, 0, 0);    // 0 are full
42     // ...
43 }
```

Figure 31.9: The Put And Get Routines

Let us set MAX=1, imagine there are two threads, a producer and a consumer.

Assume the consumer runs first & calls sem_wait(&full) (C1), decrements it to -1, and sleep.

Assume producer then runs, and call `sem_wait(&empty)`, decrement it to 0, and continue, it will call `sem_post(&full)`, incrementing full to 1, and wake the consumer.

If producer continues, it will loop to `sem_wait(&empty)`, decrement empty -1 & sleep.

If instead consumer ran, it will return from `sem_wait(&full)` consume the buffer.

In either case, we achieved desired behaviour.

This will work for more threads as well.

Lets now set `MAX > 1`, assume multiple producers & consumers.

This causes a race condition in `get()` & `put()`.

OK, let's understand the issue. Imagine two producers (Pa and Pb) both calling into `put()` at roughly the same time. Assume producer Pa gets to run first, and just starts to fill the first buffer entry (`fill=0` at Line F1). Before Pa gets a chance to increment the `fill` counter to 1, it is interrupted. Producer Pb starts to run, and at Line F1 it also puts its data into the 0th element of buffer, which means that the old data there is overwritten! This action is a no-no; we don't want any data from the producer to be lost.

A Solution: Adding Mutual Exclusion

We have to add locks around them as:

```
1 void *producer(void *arg) {
2     int i;
3     for (i = 0; i < loops; i++) {
4         sem_wait(&mutex);           // Line P0 (NEW LINE)
5         sem_wait(&empty);           // Line P1
6         put(i);                     // Line P2
7         sem_post(&full);             // Line P3
8         sem_post(&mutex);           // Line P4 (NEW LINE)
9     }
10 }
11
```



```

12 void *consumer(void *arg) {
13     int i;
14     for (i = 0; i < loops; i++) {
15         sem_wait(&mutex);           // Line C0 (NEW LINE)
16         sem_wait(&full);            // Line C1
17         int tmp = get();            // Line C2
18         sem_post(&empty);           // Line C3
19         sem_post(&mutex);           // Line C4 (NEW LINE)
20         printf("%d\n", tmp);
21     }
22 }

```

Figure 31.11: Adding Mutual Exclusion (Incorrectly)

But, now, there is a Deadlock.

Avoiding Deadlock

Imagine two threads, one producer & one consumer.

The consumer runs first, acquires the lock, then calls `sem_wait(&full)`, and sleeps as buffer is empty.

Now, the producer runs, tries to acquire the lock but finds it held, and sleeps.

At Last, A working solution

Move the lock to be around the critical section.

```

1  void *producer(void *arg) {
2      int i;
3      for (i = 0; i < loops; i++) {
4          sem_wait(&empty);           // Line P1
5          sem_wait(&mutex);           // Line P1.5 (lock)
6          put(i);                     // Line P2
7          sem_post(&mutex);           // Line P2.5 (unlock)
8          sem_post(&full);            // Line P3
9      }
10 }

11
12 void *consumer(void *arg) {
13     int i;
14     for (i = 0; i < loops; i++) {
15         sem_wait(&full);             // Line C1
16         sem_wait(&mutex);           // Line C1.5 (lock)
17         int tmp = get();            // Line C2
18         sem_post(&mutex);           // Line C2.5 (unlock)
19         sem_post(&empty);           // Line C3
20         printf("%d\n", tmp);

```

```
21 }  
22 }
```

Figure 31.12: Adding Mutual Exclusion (Correctly)

Reader-Writer Locks

Imagine a list of concurrent list operations including inserts & lookups.

Inserts change the state of list. So, a traditional critical section makes sense.

However, lookups simply read the data structure & as such we allow many concurrent lookups.

This special-type of lock is called Reader-Writer lock.

```
1  typedef struct _rwlock_t {  
2      sem_t lock;          // binary semaphore (basic lock)  
3      sem_t writelock;    // allow ONE writer/MANY readers  
4      int    readers;     // #readers in critical section  
5  } rwlock_t;  
6  
7  void rwlock_init(rwlock_t *rw) {  
8      rw->readers = 0;  
9      sem_init(&rw->lock, 0, 1);  
10     sem_init(&rw->writelock, 0, 1);  
11 }  
12  
13 void rwlock_acquire_readlock(rwlock_t *rw) {  
14     sem_wait(&rw->lock);  
15     rw->readers++;  
16     if (rw->readers == 1) // first reader gets writelock  
17         sem_wait(&rw->writelock);  
18     sem_post(&rw->lock);  
19 }  
20  
21 void rwlock_release_readlock(rwlock_t *rw) {  
22     sem_wait(&rw->lock);  
23     rw->readers--;  
24     if (rw->readers == 0) // last reader lets it go  
25         sem_post(&rw->writelock);  
26     sem_post(&rw->lock);  
27 }  
28  
29 void rwlock_acquire_writelock(rwlock_t *rw) {  
30     sem_wait(&rw->writelock);  
31 }  
32  
33 void rwlock_release_writelock(rwlock_t *rw) {  
34     sem_post(&rw->writelock);  
35 }
```


The above implementation is simple.

If some thread wants to update the data structure in question, it should call:

`rwlock_acquire_writelock()` to acquire the write lock, and

`rwlock_release_writelock()` to release it.

When acquiring the readlock, the reader first acquires lock and then increments the readers variable (which tracks number of readers).

If the reader is the first, then it will acquire the writelock & release the lock by calling `sem-post()`.

Thus, once a reader has acquired a readlock, more readers will be allowed to acquire the read lock too.

However, any thread that wishes to acquire the writelock will have to wait until all readers are finished; the last one to exit critical section calls `sem-post()` on "writelock" & enables the waiting writer to acquire the lock.

-The readers can easily starve the writers

What you would need to do prevent more readers from entering the lock once a writer is waiting?

```
typedef struct __rwlock_t {
    sem_t lock;          // binary semaphore (basic lock for reader_count)
    sem_t writelock;     // allow ONE writer / MANY readers
    sem_t turnstile;     // NEW: allows writers to block new readers
    int readers;         // #readers in critical section
} rwlock_t;

void rwlock_init(rwlock_t *rw) {
    rw->readers = 0;
    sem_init(&rw->lock, 0, 1);
    sem_init(&rw->writelock, 0, 1);
    sem_init(&rw->turnstile, 0, 1); // Initialize turnstile to 1
}
```

```
}

void rwlock_acquire_readlock(rwlock_t *rw) {
    // 1. Wait at the turnstile first
    sem_wait(&rw->turnstile);
    // 2. Immediately release so other readers can also pass
    sem_post(&rw->turnstile);

    sem_wait(&rw->lock);
    rw->readers++;
    if (rw->readers == 1) // first reader gets writelock
        sem_wait(&rw->writelock);
    sem_post(&rw->lock);
}

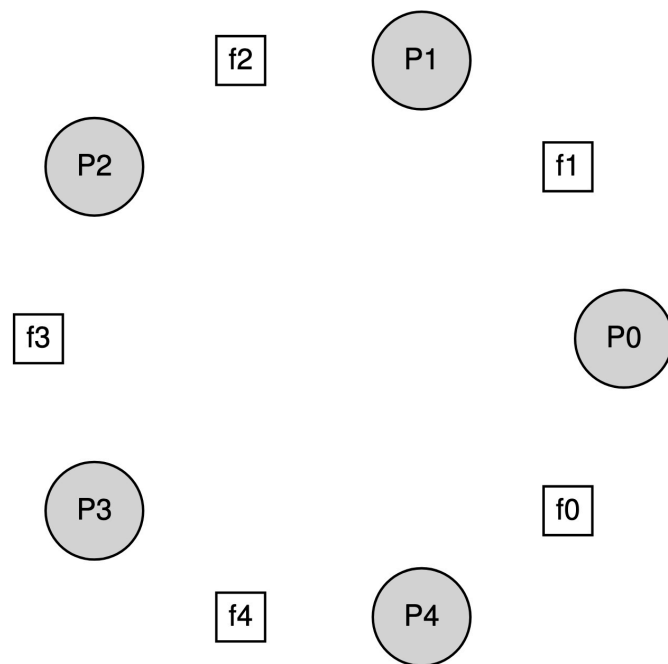
void rwlock_release_readlock(rwlock_t *rw) {
    sem_wait(&rw->lock);
    rw->readers--;
    if (rw->readers == 0) // last reader lets it go
        sem_post(&rw->writelock);
    sem_post(&rw->lock);
}

void rwlock_acquire_writelock(rwlock_t *rw) {
    // 1. Lock the turnstile so no NEW readers can enter the queue
    sem_wait(&rw->turnstile);
    // 2. Wait for the current readers to finish (the room to empty)
    sem_wait(&rw->writelock);
}

void rwlock_release_writelock(rwlock_t *rw) {
    sem_post(&rw->writelock);
    // Open the turnstile for everyone else
    sem_post(&rw->turnstile);
}
```


The Dining Philosophers

Assume there are five philosophers sitting around a table. Between each pair of philosophers is a single fork (Total 5 forks).



The philosophers each have times where they think, and don't need any forks, and times where they eat.

In order to eat, a philosopher needs two forks from both their sides.

Below is basic loop of each philosopher:

```
while (1) {  
    think();  
    get_forks(p);  
    eat();  
    put_forks(p);  
}
```

The key challenge is to write routines `get_forks()` & `put_forks()` such that there is no deadlock, starvation, and concurrency is high.

Some helper functions:

```
int left(int p) { return p; }  
int right(int p) { return (p + 1) % 5; }
```

When philosopher p wishes to refer to the fork on their left, they simply call `left(p)`. Similarly, the fork on the right of a philosopher p is referred to by calling `right(p)`; the modulo operator therein handles the one case where the last philosopher ($p=4$) tries to grab the fork on their right, which is fork 0.

Assume we have a semaphore for each fork `sem_t forks[5]`.

Broken solution

```
1 void get_forks(int p) {  
2     sem_wait(&forks[left(p)]);  
3     sem_wait(&forks[right(p)]);  
4 }  
5  
6 void put_forks(int p) {  
7     sem_post(&forks[left(p)]);  
8     sem_post(&forks[right(p)]);  
9 }
```

To acquire the forks, we simply grab a "lock" on each one. First the one on the left, then the one on the right.

When philosopher is done, he releases them.

Unfortunately, we have a **deadlock**. If each philosopher grabs the fork on their left & are context switched before they can grab the one on their right.

Then, each (when they come back) will wait for their right fork forever.

A solution: Breaking the Dependency

The simplest way to solve this is to change how forks are acquired by at least one of the philosophers.


```

1 void get_forks(int p) {
2     if (p == 4) {
3         sem_wait(&forks[right(p)]);
4         sem_wait(&forks[left(p)]);
5     } else {
6         sem_wait(&forks[left(p)]);
7         sem_wait(&forks[right(p)]);
8     }
9 }

```

As the last philosopher tries to grab right before left, the cycle is broken.

There are other “famous” problems like this one, e.g., the **cigarette smoker’s problem** or the **sleeping barber problem**.

Thread Throttling

One other simple use case for semaphores arises on occasion, and thus we present it here. The specific problem is this: how can a programmer prevent “too many” threads from doing something at once and bogging the system down? Answer: decide upon a threshold for “too many”, and then use a semaphore to limit the number of threads concurrently executing the piece of code in question. We call this approach **throttling** [T99], and consider it a form of **admission control**.

Let’s consider a more specific example. Imagine that you create hundreds of threads to work on some problem in parallel. However, in a certain part of the code, each thread acquires a large amount of memory to perform part of the computation; let’s call this part of the code the *memory-intensive region*. If *all* of the threads enter the memory-intensive region at the same time, the sum of all the memory allocation requests will exceed the amount of physical memory on the machine. As a result, the machine will start thrashing (i.e., swapping pages to and from the disk), and the entire computation will slow to a crawl.

A simple semaphore can solve this problem. By initializing the value of the semaphore to the maximum number of threads you wish to enter the memory-intensive region at once, and then putting a `sem_wait()` and `sem_post()` around the region, a semaphore can naturally throttle the number of threads that are ever concurrently in the dangerous region of the code.

How to Implement Semaphores

One subtle difference b/w zsemaphores & pure semaphores is that they don't maintain the invariant that the value of the semaphore, when negative, reflects the no. of threads waiting.

In fact, the value will never be lower than zero.

```
1  typedef struct __Zem_t {
2      int value;
3      pthread_cond_t cond;
4      pthread_mutex_t lock;
5  } Zem_t;
6
7  // only one thread can call this
8  void Zem_init(Zem_t *s, int value) {
9      s->value = value;
10     Cond_init(&s->cond);
11     Mutex_init(&s->lock);
12 }
13
14 void Zem_wait(Zem_t *s) {
15     Mutex_lock(&s->lock);
16     while (s->value <= 0)
17         Cond_wait(&s->cond, &s->lock);
18     s->value--;
19     Mutex_unlock(&s->lock);
20 }
21
22 void Zem_post(Zem_t *s) {
23     Mutex_lock(&s->lock);
24     s->value++;
25     Cond_signal(&s->cond);
26     Mutex_unlock(&s->lock);
27 }
```

} never is s->value negative.